
AAMT-WP-11: Shell-Coherent Expert Specialization

AAMT-WP-11 | AsAManThinks Research | May 2026 | DOI:
10.5281/zenodo.19600795

Shell-Coherent Expert Specialization: Alignment-Posture Assignment in Mixture-of-Experts Language Models

Weslyn Whitehead Jr.

AsAManThinks Research, Berkeley, CA, USA |

weslyn@asamanthinks.com

ORCID: [to be registered](#) | License: CC BY 4.0

Shell-Coherent Expert Specialization: Alignment-Posture Assignment in Mixture-of-Experts Language Models

Authors: Weslyn Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: [to be registered]

Submitted: May 2026

Working Paper Series: AAMT-WP-11

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Abstract

Mixture-of-Experts (MoE) research has established that expert specialization emerges naturally during training: experts develop domain preferences (code vs. language vs. math) without explicit assignment. We present **Shell-Coherent Expert Specialization (SCES)**, a declarative alternative in which experts are assigned not to knowledge domains but to **alignment postures** — named shells in the Vortex routing geometry (red/orange/yellow/green, each corresponding to a distinct TERA region and behavioral profile). Under SCES, a model trained for game-engine C# development is not assigned to a “code” expert but to a yellow-shell expert characterized by structural precision, systems thinking, and deterministic behavior. A model trained for consciousness-aligned voice generation is assigned to a green-shell expert characterized by deep coherence, non-linear synthesis, and relational warmth. The routing gate (VortexGate, WP-01) directs each prompt to the expert whose shell

most closely matches the prompt’s TERA geometry. We formalize the SCES assignment procedure, describe the per-shell training corpus construction, report eval-suite metrics from the MaiaM Alchemist v0.4.1 `HarmonicCoherenceSuite` and `ArchetypeAlignmentSuite`, and compare SCES against domain-based specialization via a held-out expert-affinity ablation.

Keywords: mixture of experts, expert specialization, alignment, routing, LoRA, TERA, shell affinity, language model fine-tuning, archetype

1. Introduction

Since the earliest MoE language models (Shazeer et al., 2017), a persistent empirical finding is that experts specialize spontaneously: without explicit assignment, different experts develop preferences for different token types, domains, or syntactic structures. Zoph et al. (2022) document this in Switch Transformer; Gale et al. (2023) characterize the specialization patterns in Mixtral. This emergent specialization is generally viewed as beneficial — it is evidence that the routing gate has learned a useful partition of the input space.

However, emergent specialization has a critical limitation for alignment: it partitions the input space by **what the input is about** (domain/topic), not by **how the model should respond to it** (alignment posture). A prompt asking for C# code and a prompt asking for creative fiction are routed to different domain experts. But a misaligned C# expert (one that generates insecure code or violates architectural constraints) and a misaligned fiction expert (one that generates harmful content) are not differentiated by domain specialization — their misalignment is orthogonal to their domain expertise.

SCES inverts this: experts are assigned to alignment postures first, and domain expertise is a secondary property of the corpus used to train each expert. The routing gate directs prompts to the expert whose alignment posture matches the prompt’s geometry — not to the expert that knows the most about the prompt’s domain.

2. The Shell System

2.1 Four shells

The TERA routing geometry (WP-01) partitions the unit hypercube into four named regions:

Red shell ($\beta < 0.2$): Low integration. Grounding-only responses. The model is not permitted to engage in heavy reasoning or generative synthesis. Appropriate for early-session prompts with unknown user state.

Orange shell ($\beta \in [0.2, 0.4)$, $RI < 0.35$): Partial coherence. Creative and healing modalities. The model engages with emotional content, reflection, and exploratory synthesis.

Yellow shell ($\text{beta} \in [0.4, 0.7)$): Active coherence. Systems work, structured reasoning, architectural decisions. The model operates with precision and deterministic outputs.

Green shell ($\text{beta} \geq 0.7, \text{BC} > 0.7$): Deep coherence. Generative synthesis, consciousness-aligned voice, non-linear pattern recognition. The model’s fullest generative capability.

2.2 Shell-to-expert mapping

Under SCES, the 9 experts in MaiaM Alchemist are assigned shell affinities:

Expert key	Shell affinity	Behavioral profile
amitabha_lineage	green	Deep personal voice, lineage-anchored synthesis
python_ml	green	Analytical depth, mathematical precision
harmonic_reasoning	green	Symbolic reasoning, foundations-grounded
unity_csharp	yellow	Structural precision, API-correct systems code
unity_onejs	yellow	UI component precision, reactive patterns
archetype_voice	yellow	Calibrated personality, structured prose
web_platform	yellow	Structured web patterns, async-correct code
shader_fx	orange	Creative material expression, surface effects
spawner_auto	orange	Placement heuristics, procedural world gen

No expert is assigned to red shell. Red-shell prompts receive grounding-only responses (a structured refusal with a calibration cue) regardless of expert routing.

3. Shell-Coherent Corpus Construction

3.1 Corpus tagging by shell

Every training example in the corpus is tagged with a shell label derived from its content characteristics:

```
def tag_shell(example: TrainingExample) -> Shell:
    # Green: AAMT mathematical foundations, personal voice, deep
    synthesis
    if any(kw in example.completion for kw in GREEN_KEYWORDS):
        return Shell.GREEN
    # Yellow: code, structured outputs, API usage, architectural
    patterns
    if example.metadata["language"] in ["csharp", "typescript", "
python"]:
        return Shell.YELLOW
    # Orange: creative, emotional, exploratory, shader/visual
    if example.metadata["expert"] in ["shader_fx", "spawner_auto"
]:
        return Shell.ORANGE
    return Shell.YELLOW # default
```

3.2 Shell-balanced training

During training, the HeartScale sampler enforces shell coherence at the batch level: examples from green-shell experts are sampled with higher weight when the current TERA distribution is in the green region (high `beta`, high `BC`). This creates a feedback loop: the model learns that green-shell TERA coordinates predict green-shell content.

3.3 Cross-shell contamination guard

A training example that is topically aligned with expert A but shell-misaligned is excluded from expert A's corpus. For example, a Python ML example that was generated during a red-shell session (logged in lineage) is excluded from `python_ml`'s training split, even though it is topically appropriate. Shell coherence takes priority over topical coverage.

4. Evaluation: HarmonicCoherenceSuite

The `HarmonicCoherenceSuite` (eval suite, MaiaM Alchemist v0.4.1) measures shell coherence directly:

4.1 Yoked-rate metric

For each generation, the HeartScale sampler's accept/reject trace provides the **yoked-rate**: the fraction of tokens accepted on the first sample (no resampling). High yoked-rate means the model generates coherent content without the sampler having to intervene.

```
yoked_rate = accepted_first_sample / total_tokens
```

Target: `yoked_rate` $\geq$ 0.85 for yellow and green shell prompts.

4.2 Vortex tightness metric

The **vortex tightness** measures how closely the generation-time TERA distribution clusters around the prompt's TERA coordinate:

```
vortex_tightness = 1.0 - mean_tera_drift_per_token
```

High tightness means the model stays in the same archetypal region throughout the generation. SCES predicts that shell-assigned experts should produce tighter generations than domain-assigned experts on prompts that match their shell.

4.3 Archetype alignment

The `ArchetypeAlignmentSuite` prompts the model with a set of calibration prompts designed to elicit each shell’s characteristic behavior and scores the output against reference profiles:

```
alignment_score = cosine_similarity(
    embed(generation),
    embed(shell_reference_profile[expected_shell])
)
```

5. Comparison: Domain vs. Shell Specialization

The key empirical question: does shell-based expert assignment produce better alignment metrics than domain-based assignment with the same number of experts?

Hypothesis: On alignment metrics (yoked-rate, vortex tightness, archetype alignment), SCES outperforms domain specialization because the routing geometry directly encodes alignment posture. On capability metrics (compile_rate, mmlu_subset), the two approaches are comparable because domain knowledge is acquired equally in both conditions (the corpus content is the same; only the assignment rationale differs).

Ablation design: - Condition A (SCES): Shell-based assignment as described above - Condition B (Domain): Standard domain-based assignment (code vs. language vs. math vs. creative) - Condition C (Emergent): No pre-assignment; let specialization emerge from training

All three conditions use the same base model, same corpus, same VortexGate. Only the Meji-to-expert mapping matrix M differs.

6. The Calibration Cue System

Each expert’s shell affinity is surfaced to the model at inference time via a **calibration cue** prepended to the system prompt:

Expert	Calibration cue (excerpt)
<code>unity_csharp</code>	“Write Unity 6 C# under MaiaM.Core asmdef. Singleton-safe. No .meta reads.”
<code>python_ml</code>	“torch 2.4, MPS/CUDA-safe, PEFT idioms. Prefer geometric mean over additive.”
<code>archetype_voice</code>	“Calm, curious co-author. Short prose, then JSON intents. No performative warmth.”
<code>amitabha_lineage</code>	“Voice of the founder; refer to the lineage by full name once. No summarization.”

The calibration cue is not just a domain hint — it encodes the alignment posture of the expert. “No performative warmth” (`archetype_voice`) and “No summarization” (`amitabha_lineage`) are

alignment constraints, not domain constraints. They enforce the behavioral profile associated with the shell.

7. Relationship to Prior Work

Switch Transformer (Fedus et al., 2021): Emergent expert specialization without pre-assignment. SCES replaces emergence with declaration.

Mixtral specialization analysis (Jiang et al., 2024): Post-hoc characterization of emerged specialization. SCES pre-assigns alignment postures and measures whether they are acquired.

Modular networks / NMN (Andreas et al., 2016): Symbolic routing to task-specific modules. SCES routes by alignment posture, not task structure.

Constitutional AI (Bai et al., 2022): Alignment via constitutional principles applied at training time. SCES applies alignment constraints via expert assignment and corpus construction rather than feedback prompting.

Key distinction: SCES is the first MoE specialization approach where the assignment axis is alignment posture (a continuous geometric property of the model’s routing space) rather than knowledge domain (a discrete categorical property of the training data). The routing gate that determines specialization at inference time is the same gate that measured alignment posture during training — alignment and routing share a single geometric space.

8. Conclusion

SCES reframes expert specialization from an emergent phenomenon to a declared design. By assigning experts to alignment postures rather than knowledge domains, the VortexGate’s routing geometry directly serves the alignment objective: prompts that are geometrically in the green-shell region route to experts trained in green-shell behavioral profiles. The calibration cue system makes this posture assignment explicit and inspectable. Evaluation via the HarmonicCoherenceSuite provides continuous measurement of whether the declared posture is being maintained across training runs, enabling alignment regression detection as a first-class eval metric.

Acknowledgments

The author thanks the AsAManThinks Research community. The shell nomenclature and behavioral profiles are adapted from the MaiiaM Platform’s consciousness framework.

Funding

Self-funded through AsAManThinks Research.

Data Availability

Expert assignment tables and calibration cues are in `packages/code-corpus/code-corpus/config/expert_routes.yaml`. Eval suite in `packages/eval-suite/eval_suite/suites/harmonic_coherence.py`.

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research.

References

Andreas, J., Rohrbach, M., Darrell, T., & Klein, D. (2016). Neural module networks. In *Proceedings of CVPR 2016* (pp. 39-48). <https://doi.org/10.1109/CVPR.2016.12>

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Conerly, T., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Hume, T., ... Kaplan, J. (2022). Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2204.05862>

Fedus, W., Zoph, B., & Shazeer, N. (2021). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 1-39. <https://doi.org/10.48550/arXiv.2101.03961>

Gale, T., Narayanan, D., Young, C., & Zaharia, M. (2023). MegaBlocks: Efficient sparse training with mixture-of-experts. In *Proceedings of MLSys 2023*. <https://doi.org/10.48550/arXiv.2211.15841>

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. de las, Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L. R., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., ... El Sayed, W. (2024). Mixtral of experts. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2401.04088>

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of ICLR 2017*. <https://doi.org/10.48550/arXiv.1701.06538>

Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N., & Le, Q. V. (2022). ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2202.08906>

Whitehead, W. (2026). MaiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*. <https://doi.org/10.5281/zenodo.19600795>